

AI Assignment #1 — Task 2 Report

Course: Artificial Intelligence (5th Semester)

Instructor: Sir Touqeer Abbas

Date: May 23, 2026

Student Name: Manahil Mahmood

Roll Number: 0000674952

1. Introduction & Objective

The primary objective of Task 2 was to design, develop, and deploy a fully functional AI-powered agent utilizing a free Large Language Model (LLM) API framework. The core requirement specified creating an interactive web application where users can seamlessly converse with an AI agent that operates under a strict, predefined persona and addresses domain-specific queries.

For this task, I successfully developed the "AI Code Helper" — an interactive, friendly, and specialized Python programming tutor designed to assist beginner to intermediate learners.

2. Technology Stack

To ensure a lightweight, fast, and highly responsive web application, the following technology stack was systematically selected and integrated:

- **Programming Language:** Python 3.x
- **Web Framework:** Streamlit (leveraged for rapid UI component development and chat-style layout engineering)
- **LLM API Gateway:** Groq Inference Engine (chosen for its ultra-fast inference speed and optimized processing of open-source models)
- **LLM Model:** Llama 3 (8B Parameters execution)
- **Environment Management:** python-dotenv (utilized for secure, industry-standard API key isolation and management)

3. Agent Architecture & Behavior

The AI agent's behavior, personality boundary, and domain limitations were tightly controlled using a tailored System Prompt. This engineering technique prevents model hallucination, restricts tangential deviations into unrelated topics, and keeps the agent strictly anchored to its pedagogical role.

System Prompt Definition:

"You are a friendly and helpful AI Python Tutor. Your job is to: 1. Explain Python concepts in simple, beginner-friendly language. 2. Debug Python code when users paste it with errors. 3. Provide short and clear code examples. 4. Answer questions about Python libraries. Always be encouraging!"

Query Handling Capabilities

The architecture is structured to interpret and resolve at least 5 distinct categories of core queries with high contextual efficiency:

1. **Concept Explanation:** Simplifying foundational topics (e.g., *"What is a for loop and how does it iterate?"*)
2. **Code Debugging:** Analyzing syntax/runtime errors and offering fixes (e.g., *"Why is this code giving an error: `prnt('hello')`?"*)
3. **Code Generation:** Writing modular, well-commented functions based on user requirements (e.g., *"Write a Python function to add two numbers."*)
4. **Library Overviews:** Clarifying the purpose of external data packages (e.g., *"What is the pandas library used for in data science?"*)
5. **Best Practices & Conventions:** Guiding users on code style standards (e.g., *"How should I properly name variables using PEP 8 rules?"*)

4. Implementation Details

A. Backend & API Integration

The core logic is orchestrated via the official groq Python SDK. To ensure continuity in user conversations, session-based state management is implemented using Streamlit's `st.session_state`. When a user submits a prompt, the absolute chat history—prepended by the structural System Prompt—is packaged and transmitted sequentially to the Groq API. This enables the Llama 3 model to accurately preserve dialogue context and return coherent, human-like responses.

B. Security & Environment Variables

In strict compliance with software development best practices and assignment guidelines, the sensitive Groq API key is completely isolated from the runtime source files. The system loads keys dynamically at initialization from a protected `.env` file using the `python-dotenv` library. Furthermore, a non-sensitive `.env.example` template has been published in the repository root to guide external setup without exposing confidential deployment credentials.

C. Frontend & User Interface Design

Streamlit was purposefully selected over traditional web frameworks (like Flask or Django) to facilitate a full-stack experience without requiring independent HTML, CSS, or JavaScript configurations. The interface offers an intuitive, modern user experience including:

- A streamlined, professional chat canvas (User inputs anchored to the right, AI responses rendered cleanly on the left).
- An interactive native chat interface text input box.
- An asynchronous "Thinking..." visual spinner to indicate active API processing and token generation periods.
- Rich Markdown support providing clear background formatting for blocks of code and technical text.

5. Testing & Results

The application deployment was validated in a cloud environment (Google Colab) mapped to an external access tunnel using Cloudflare Tunnel (cloudflared).

During intensive validation testing, the web app functioned flawlessly. The combination of the Llama 3 model with Groq's hardware acceleration delivered processing speeds with a minimal latency of under 2 seconds per response token pool. Most importantly, the agent tightly adhered to its behavioral guards—providing rich, accurate Python tutorials while politely declining to answer off-topic queries outside the scope of computer programming.

Ask me anything about Python!

 You

what is python

 CodeHelper AI

Introduction to Python

Python is a high-level, interpreted programming language that is widely used for various purposes such as:

- * Web development
- * Scientific computing
- * Data analysis
- * Artificial intelligence
- * Automation
- * Education

Key Features of Python

- * **Easy to learn:** Python has a simple syntax and is relatively easy to learn, making it a great language for beginners.
- * **Versatile:** Python can be used for a wide range of applications, from web development to scientific computing.
- * **Large community:** Python has a large and active community, which means there are many resources available for learning and troubleshooting.
- * **Extensive libraries:** Python has a vast collection of libraries and frameworks that make it easy to perform various tasks, such as data analysis, machine learning, and web development.

How Python Works

Figure 1: AI agent explaining a python concept.

 You
Debug this code: `prnt("Hello World")`

 CodeHelper AI
Debugging the Code
=====

The code you provided is:

```
prnt("Hello World")
```

The issue with this code is a **typo** in the `print` function. The correct function name is `print`, not `prnt`.

Corrected Code
=====

Here's the corrected code:

```
print("Hello World")
```

Explanation of the Error
=====

In Python, the `print` function is used to output text to the screen. When you try to use `prnt` instead of `print`, Python will raise a `NameError` because it doesn't recognize `prnt` as a valid function.

Error Message
=====

If you try to run the original code, you'll get an error message like this:

```
NameError: name 'prnt' is not defined
```

Figure 2: AI AGENT debugging a code snippet.

 You
Write a Python function to check if a number is prime

 CodeHelper AI
Prime Number Checker Function
=====

Here's a Python function that checks if a number is prime:

```
def is_prime(n):
    """
    Checks if a number is prime.

    Args:
        n (int): The number to check.

    Returns:
        bool: True if the number is prime, False otherwise.
    """
    if n <= 1:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True
```

Figure 3: AI Agent checking if number is prime.

6. Conclusion

The implementation of the "AI Code Helper" successfully provided practical exposure to integrating modern Large Language Model APIs into an end-to-end user application. Utilizing Streamlit significantly reduced frontend overhead, while Groq provided efficient, low-latency inference speeds. Deploying key protections via environment variables and maintaining context via session state frameworks illustrates a highly secure, reliable, and scalable blueprint for engineering modern AI-driven solutions.